

Information Technologies for Industrial Engineers

Docker and Self Hosted Database

Motivation

You have used Supabase before, but we will learn how to **self host** a database.

Why self host a database?

- Control over data and security
- Customization and flexibility
- Cost-effective in the long run
- No dependency on third-party services

How to self host a database?

- Choose a database software (e.g., MySQL, PostgreSQL, MongoDB).
- Set up a server (local machine, cloud server, or dedicated server).
- **Install the database software.**
- Configure the database settings (user accounts, permissions, etc.).
- Create and manage databases and tables.

Installing method

- Native installation
- **Docker container**

Native installation

- You do this as if you want to install another software on your computer. You can download the installer from the official website and follow the installation instructions.
- This method is easier to get started.
- However, your environment is not isolated, and it may cause conflicts with other software or dependencies.
 - *"Works on my computer, but not on yours."*
- This is not recommended for production use.
 - Docker is a better option for production use.

What is Docker?

- Docker is a platform (and a company) that allows you to package and run applications in isolated environments called **containers**.

What is a container?

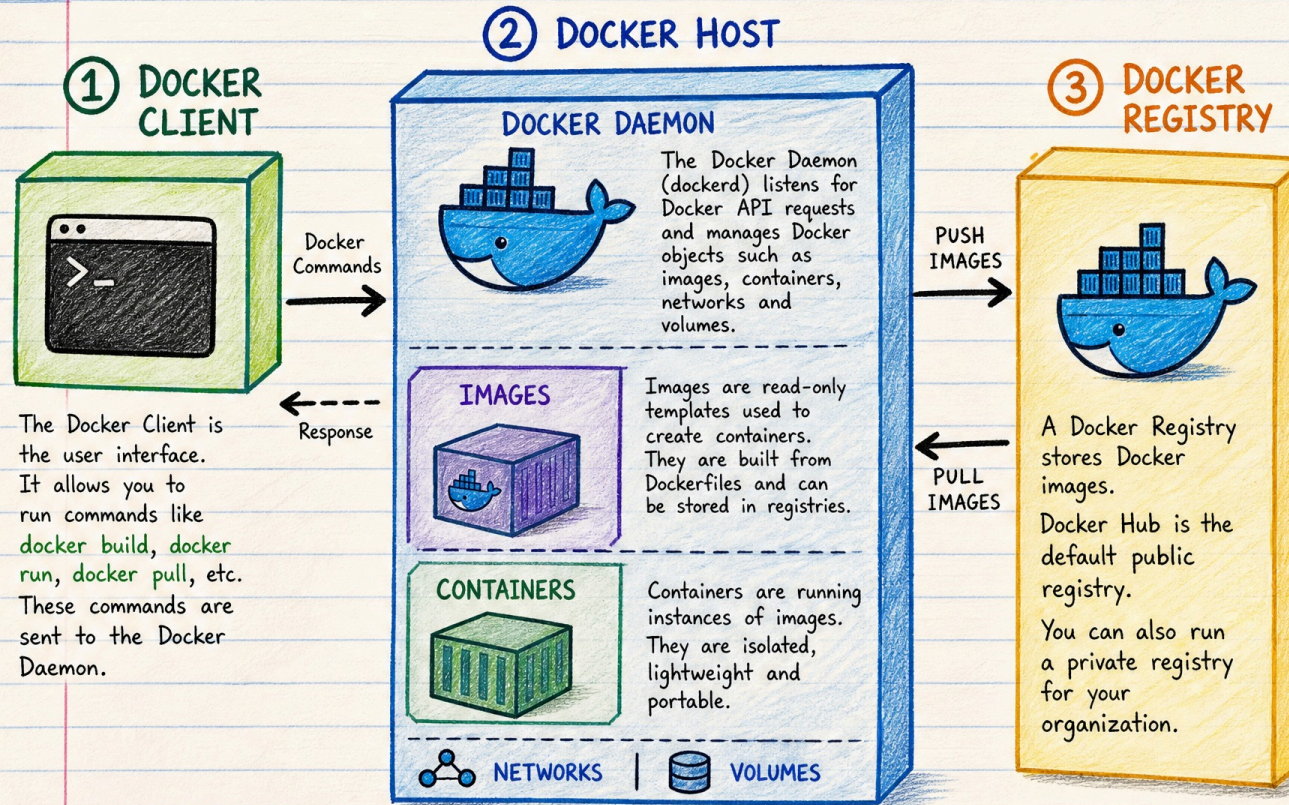
- A container is a lightweight, standalone, and executable package that includes everything needed to run an application: code, runtime, system tools, libraries, and settings.
- Containers are **isolated** from each other and the host system, ensuring that they run consistently across different environments.
- You can think about container as another computer that runs on your computer.
 - It has its own operating system, file system, and network stack.

DOCKER ARCHITECTURE



Docker uses a **client-server architecture**.

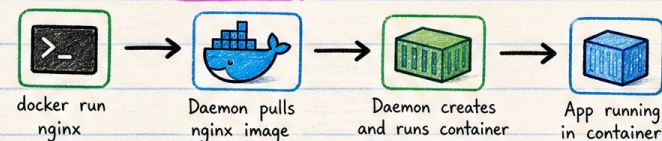
The client talks to the **Docker daemon**, which manages images, containers and networks.



KEY POINTS

- ★ Client sends commands to the Daemon.
- ★ Daemon builds, runs and manages containers.
- ★ Images are stored in Registries.
- ★ Containers run on the Host.

EXAMPLE FLOW



TIP: Think of Docker as a shipping system for your applications!



Source

Docker Vocabulary

- **Image:** A read-only template that contains the application and its dependencies. Images are used to create containers.
- **Container:** A running instance of an image. Containers are isolated from each other and the host system.
- **Registry:** A storage and distribution system for Docker images. Docker Hub is the default public registry, but you can also use private registries.

Analogy

Concept	Analogy
Image	installer file (e.g., <code>.exe</code> , <code>.dmg</code> , <code>.deb</code>)
Container	installed application (e.g., <code>C:\Program Files\MyApp</code> , <code>/Applications/MyApp.app</code> , <code>/usr/local/bin/myapp</code>)
Registry	app store (e.g., Microsoft Store, Mac App Store, Snap Store)

Get started with Docker

Docker Requirements

- 8 Gb of RAM (4 GB minimum)
- 6 GB of free disk space
- Ensure Hardware Virtualization is enabled in your BIOS/UEFI (usually enabled by default on modern laptops).

Installation

- Install Docker Desktop
 - <https://www.docker.com/products/docker-desktop/>
 - *(Windows)* `choco install docker-desktop`
 - *(macOS)* `brew install --cask docker`
- Notes
 - *(Windows)* Choose `Use WSL 2 instead of Hyper-V` option
 - *(macOS)* Download the correct installer based on your chip (Apple Silicon or Intel).

Hello World with Docker

- Run the following command to check if Docker is installed correctly:

```
docker run hello-world
```

- The `hello-world` image is a simple image that prints a message to the console. If you see the message, it means Docker is working correctly.
- Visit https://hub.docker.com/_/hello-world to learn more about the `hello-world` image.

Web-server with Docker

```
docker run -d -p 8080:80 nginxdemos/hello
```

- The `-d` flag runs the container in detached mode (in the background).
- The `-p` flag maps port 8080 on the host to port 80 in the container.
- You can access the web server by opening your browser and going to <http://localhost:8080>.
- You can also change the html file in `"/usr/share/nginx/html"` in the container.

Node.js with Docker

```
docker run -d -p 5173:5173 node:lts-alpine sleep infinity
```

- The `sleep infinity` command keeps the container running indefinitely, allowing you to interact with it.
 - Without this command, the container would exit immediately after starting, as there would be no process to keep it alive.

Run Vite Project in Docker

- `npm install -g pnpm`
- `pnpm create vite@latest`
- `cd vite-project`
- `pnpm dev --host 0.0.0.0`

Without the `--host 0.0.0.0` option, the Vite development server would only be accessible from within the container, and you wouldn't be able to access it from your host machine.

Docker Commands

- `docker ps` - List running containers
- `docker ps -a` - List all containers (running and stopped)
- `docker images` - List all images
- `docker stop <container_id>` - Stop a running container
- `docker start <container_id>` - Start a stopped container
- `docker image rm <image_id>` - Remove an image

Useful Utilities

- `docker image prune` - Remove unused images
- `docker container prune` - Remove stopped containers
- `docker volume prune` - Remove unused volumes
- `docker system prune` - Remove unused data (images, containers, volumes, networks)

Database with Docker (PostgreSQL)

- Windows

```
docker run -d \  
  --name my-postgres \  
  -e POSTGRES_PASSWORD=1234 \  
  -v pgdata:/var/lib/postgresql \  
  -p 5432:5432 \  
  postgres:latest
```

- Mac

```
docker run -d \  
  --name my-postgres \  
  -e POSTGRES_PASSWORD=1234 \  
  -v pgdata:/var/lib/postgresql \  
  -p 5432:5432 \  
  postgres:latest
```

Option	Description
<code>-d</code>	runs the container in detached mode (in the background).
<code>--name my-postgres</code>	gives the container a name for easier reference.
<code>-e POSTGRES_PASSWORD=1234</code>	sets the environment variable for the PostgreSQL password.
<code>-v</code> <code>pgdata:/var/lib/postgresql</code>	creates a named volume <code>pgdata</code> to persist the database data.
<code>-p 5432:5432</code>	maps port 5432 on the host to port 5432 in the container.

Docker volume

- A Docker volume is a persistent storage mechanism that allows you to store data outside of the container's filesystem.
- This means that even if the container is deleted or recreated, the data in the volume will remain intact.
- Very important for databases, as you don't want to lose your data when the container is removed or updated.

Where is the data stored?

Windows

```
\\wsl.localhost\docker-desktop\mnt\docker-desktop-disk\data\docker\volumes
```

Mac

```
/var/lib/docker/volumes
```

Do not modify the data in the volume directly, as it may cause data corruption.

User management

- By default, PostgreSQL creates a user called `postgres` with superuser privileges.
- You can create additional users and manage their permissions using SQL commands.

We will not be covering user management in this course, but you should be aware that using the default `postgres` user for your applications **is not recommended for production use**. Instead, create a new user with limited privileges.

Database client: Dbeaver

- Install DBeaver Community Edition (CE)
 - <https://dbeaver.io/download/>
 - (*Windows*) `choco install dbeaver-ce`
 - (*macOS*) `brew install --cask dbeaver-community`

Connection

Connection "postgres (1)" configuration

PostgreSQL
PostgreSQL standard driver

> Connection settings
General
Metadata
Errors and timeouts
Data Transfer
> Data Editor
> SQL Editor

Main Advanced Driver properties + SSH, SSL, ... No profile

Server

Connect by: ☒ Host ☐ URL

URL: jdbc:postgresql://localhost:5432/postgres

Host: localhost Port: 5432

Database: postgres ☒ Show all databases

Authentication

Authentication: Username/password

Username: postgres

Password: ☒ Save password

[Connection variables information](#) [PostgreSQL](#)

Driver name: PostgreSQL Driver Settings Driver license

Test Connection ... OK Close

Create a database

- By default, there is already a database called `postgres`. You can create a new database by running the following SQL command in DBeaver:

```
CREATE DATABASE my_db;
```

Make sure to refresh the database navigator to see the new database in the database navigator.

Create a table

- Make sure that you change the database to `my_db` (top bar) before running the following SQL command:

```
CREATE TABLE IF NOT EXISTS
todos (
  id serial PRIMARY KEY,
  title VARCHAR(50) NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

To see the table, expand the `my_db` / `Schemas` / `public` / `Tables` in the database navigator.

Create a row

```
INSERT INTO  
  todos (title)  
VALUES  
  ('First Todo'),  
  ('Second Todo'),  
  ('Third todo');
```

Delete rows

```
DELETE FROM  
  todos  
WHERE  
  id = 1;
```

or

```
DELETE FROM  
  todos  
WHERE  
  title = 'First Todo';
```

Update rows

```
UPDATE
  todos
SET
  title = 'Updated Todo'
WHERE
  id = 2;
```

Clear all rows

```
DELETE FROM  
  todos;
```

Accessing the database from node.js

Setup

- Create a new folder
- Open VSCode in that folder
- `pnpm init`
- `pnpm add pg`
- `pnpm add -D @types/pg tsx typescript`
 - `pnpm approve-builds`

pg

- The `pg` package is a PostgreSQL client for Node.js. It allows you to connect to a PostgreSQL database, execute queries, and manage transactions.
- It is very popular.

Main code

- `./src/index.ts`

```
import { Pool } from "pg";

// Configure the connection details
const pool = new Pool({
  user: "postgres", // Your PostgreSQL username
  host: "localhost", // Server hosting your database
  database: "my_db", // Name of your database
  password: "1234", // Your database password
  port: 5432, // Default PostgreSQL port
});
```

`pool` is a connection pool that manages multiple connections to the database. It allows you to reuse connections and improve performance.

Querying the database

```
const res1 = await pool.query("SELECT * from todos;");  
console.log(res1.rows);  
  
// Close the database connection, put at the end  
await pool.end();
```

Running the code

- To run type `pnpm exec tsx src/index.ts` in the terminal.
- Even better, create a script in `package.json`:

```
{  
  "scripts": {  
    "start": "tsx src/index.ts"  
  }  
}
```

Then you can run `pnpm start` to run the code.

Query a specific todo by ID

```
const todoId = 1;  
const res2 = await pool.query("SELECT * from todos WHERE id = $1;", [todoId]);  
console.log(res2.rows);
```

Notice that we use `$1` as a placeholder for the parameter. This is a good practice to prevent SQL injection attacks.

Query to insert a new todo into the database

```
const newTodo = { title: "New Todo" };
const res3 = await pool.query(
  "INSERT INTO todos (title) VALUES ($1) RETURNING *;",
  [newTodo.title],
);
console.log(res3.rows);
```

Notice that we use `RETURNING *` to return the newly inserted row. This is useful if you want to get the ID of the new todo.

Query to delete a todo by ID

```
const deleteTodoId = 1;  
const res4 = await pool.query("DELETE FROM todos WHERE id = $1 RETURNING *;", [  
  deleteTodoId,  
]);  
console.log(res4.rows);
```

Query to update a todo by ID

```
const updatedTodo = { id: 2, title: "Updated Todo" };
const res5 = await pool.query(
  "UPDATE todos SET title = $1 WHERE id = $2 RETURNING *;",
  [updatedTodo.title, updatedTodo.id],
);
console.log(res5.rows);
```

Notice that we use 2 placeholders `$1` and `$2` for the parameters.

Query to clear all todos

```
const res6 = await pool.query("DELETE FROM todos RETURNING *;");  
console.log(res6.rows);
```